

Categorical Grammar: Syntax as Algebraic Composition and Proof

Categorical Grammar: Syntax as Algebraic Composition and Proof I

Where we are in the argument. ?? built the case category — a static snapshot of licensed role-to-role morphisms. This chapter equips the framework with a *compositional syntax* on top of that static layer: Lambek pregroup types let each word carry its own combinatory potential, and cup/cap contractions reduce a parallel tensor of word-boxes to the single sentence type s , so a sentence becomes a proof of well-typedness rendered graphically as a string diagram.

Each Word Is Its Own Grammar Rule I

Traditional generative grammar computes sentence structure using top-down phrase-structure rules that recursively combine constituents. Categorical grammar inverts this perspective: rather than specifying abstract construction rules, it assigns each lexical item a dedicated *type* that encodes its combinatory potential. For example, a transitive verb like “chases” is not loosely defined as “a word requiring a subject and object,” but specifically assigned the algebraic type $(n \backslash s) / n$ —an active function that categorically demands a noun to its right (the object) and a noun to its left (the subject) to yield a complete sentence.

Lambek's Residuation Law: Consuming and Producing Types in Linear Order I

Lambek [Lambek1958mathematics] laid the algebraic foundations by introducing a *residuated* structure on syntactic types. He defined two binary operations—the left residual $\backslash$ and the right residual $/$ —derived from the multiplicative connective $\otimes$ (concatenation). The fundamental axiom governing this system is the residuation law:

$$A \otimes B \leq C \quad \Longleftrightarrow \quad A \leq C/B \quad \Longleftrightarrow \quad B \leq A \backslash C \quad (1)$$

Lambek's Residuation Law: Consuming and Producing Types in Linear Order II

This law brilliantly captures the fundamental duality of syntax: a verb of type $(n \backslash s) / n$ actively *consumes* available noun arguments to *produce* a well-formed sentence. The resulting **Lambek calculus** operates as a non-commutative intuitionistic linear logic—non-commutative because strict word order dictates meaning, and linear because the derivation logically consumes each lexical resource exactly once.

Lambek's Residuation Law: Consuming and Producing Types in Linear Order III

Pregroups Unlock Compact Closure: The Bridge to DisCoCat String Diagrams

Lambek [-@lambek2004fine] later simplified the framework to **pregroup grammars**, where each type a has a left adjoint a^l and a right adjoint a^r satisfying:

$$a^l \cdot a \leq 1 \leq a \cdot a^l \quad a \cdot a^r \leq 1 \leq a^r \cdot a \quad (2)$$

Coecke, Sadrzadeh, and Clark revolutionized this formal structuralism in 2010 with the **DisCoCat** (Distributional Compositional Categorical) model [-@coecke2010mathematical]. DisCoCat mathematically unifies categorial grammar with distributional semantics by defining strong monoidal functors mapping pregroup grammars directly into vector spaces. As Duneau emphasizes, this approach “allows the meaning of a sentence to be computed as a function of both the distributional meaning of the words involved, as well as its grammatical form” [-@duneau2021parsing]. By proving that pregroups and

Cups, Caps, and Joyal–Street: How Wires Prove Grammaticality I

The pivotal connection between categorial grammar and visualization comes from Joyal and Street [-@joyalstreet1991geometry], who proved that morphisms in monoidal categories can be faithfully represented as *string diagrams*—planar graphs where:

- ▶ **Wires** represent types (objects of the category)
- ▶ **Boxes** represent words or operations (morphisms)
- ▶ **Vertical composition** represents sequential application
- ▶ **Horizontal juxtaposition** represents the tensor product (concatenation)
- ▶ **Cups and caps** represent the contraction/expansion of a pregroup

Cups, Caps, and Joyal–Street: How Wires Prove Grammaticality II

2 surveys three syntactic structures side by side (progression from intransitive to passive); 1 shows the same transitive sentence rendered by our native matplotlib pipeline—a direct computational output confirming word-box layout, wire routing, and cup-contraction geometry independently of DisCoPy; and 3 demonstrates that type-logical structure remains invariant across diverse surface word orders (SVO, free, SOV).

Cups, Caps, and Joyal–Street: How Wires Prove Grammaticality, III

DisCoCat: "Alice chases Bob"

